

Day 3

주제 · 루프 엔지니어링

반복을 설계하는 사람이 AI를 이끈다



랄프 루프

목표 달성까지 반복 실행하는 루프



시스템 루프

루프 자체를 더 효율적으로 개선하는 루프

DAY 3

루프 엔지니어링

루프가 모두의 핵심 역량이 된 이유

우리 일은 늘 '루프'였음

01 **AI 이전에도 우리 일은 '루프'로 설명할 수 있었음**

다만 루프라는 용어를 잘 쓰진 않음 · 애자일 이터레이션 · 린스타트업 Build-Measure-Learn

02 **그런데 루프 설계는 보통 특수 직군·상위 리더십의 몫이었음**

개인의 일상 관심사는 아니었음

그런데 지금은

각 개인이 여러 에이전트를 이끄는 'AI 리더십'을 가져야 함
루프 설계가 모두의 핵심 역량이 됨

오늘 다룰 주제

01



랄프 루프

목표 달성까지 반복 실행하는 루프

02



시스템 루프

루프 자체를 더 효율적으로 개선하는 루프

AI 리더십

여러 에이전트를 이끄는 리더로서 - 오늘 스스로에게 던질 두 질문

Q1



AI가 좋은 결과를 낼 수 있는 **좋은 목표**를 제시해줄 수 있는가?

Q2



루프 관점으로 사람과 AI의 협력을 설계할 수 있는가?

DAY 3

라프 루프

단순 무식한 반복이 이긴다

문제 의식

AI 코딩 에이전트는 컨텍스트가 차면 **성능이 급락** 함

어제 다룬 drift · 오염

그렇다면

기억을 비우고 **처음부터 다시** 시키면
되지 않을까?

공개된 "기법"의 실체

호주 개발자 **Geoffrey Huntley**가 2025년 6월 밋업 시연 → 7월 블로그 공개

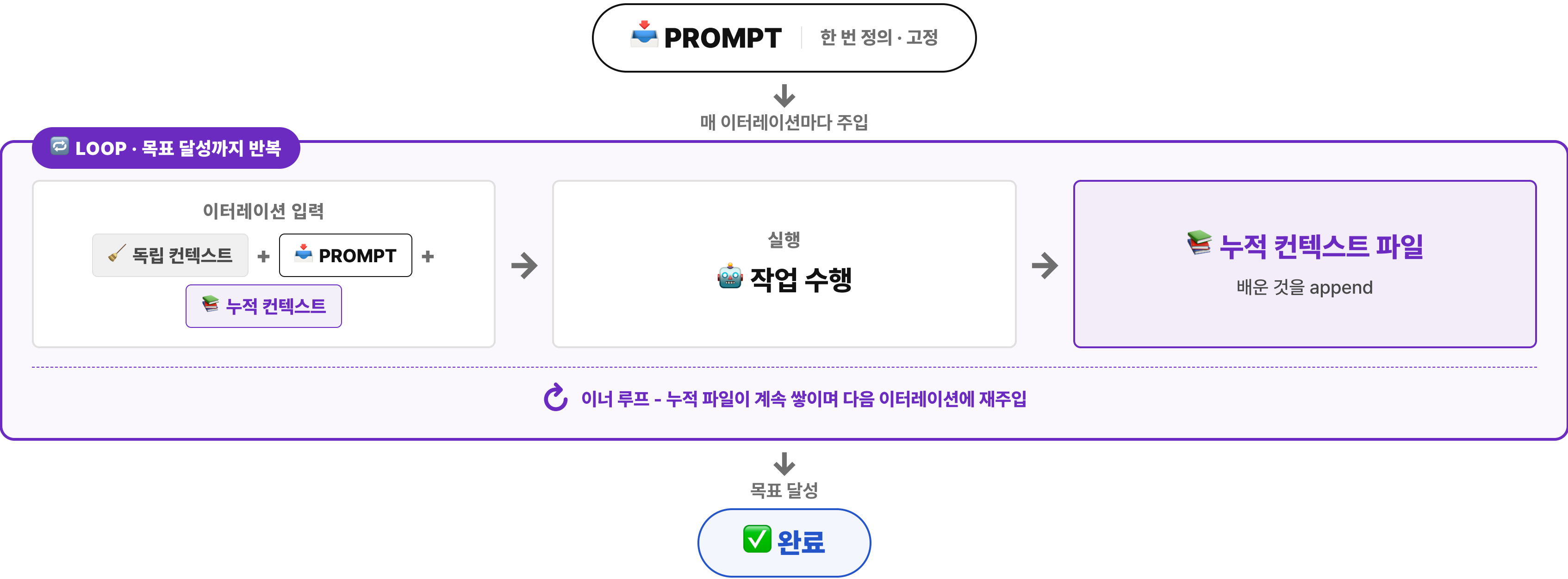
[3] Huntley · ghuntley.com/ralph

bash

```
$ while :; do cat PROMPT.md | (에이전트); done
```

👉 그냥 무한 bash 루프 - 충격 포인트는 이렇게 단순한 게 실제로 통한다는 것

랄프 루프의 흐름



핵심 통찰

- 01 진행 상황을 LLM 기억이 아니라 파일·git에 저장
- 02 컨텍스트가 오염되면 세션을 버리고, 새 에이전트가 파일에 쌓인 결과를 보고 이어감

👉 Day2의 컨텍스트 오염 문제에 대한 해법

이름의 유래

심슨의 캐릭터 **'랄프 위검'** - 멍청하지만 천진하게 끈질김

역설

"똑똑한 한 방"이 아니라 "단순 무식한 반복"이 이긴다

단순 무식한 반복의 성공은 LLM 근간인 트랜스포머(딥러닝)도 마찬가지임

확인된 개념 활용 사례

01



뷰티 ML 모델 개선

autosearch 개념을 차용해 모델을 반복 개선

02



제품 프로토타이핑

복잡한 개념을 스크린샷을 매번 찍어 이터레이션
별 비교

03



nanochat 비용 최적화

안드레 카파시 nanochat의 학습 비용을
autoresearch로 개선

Sellop 샵빌더 사례

01 개요

자연어로 상점을 **실시간 수정**하는 기능 (예: "배너를 더 크게 바꿔줘")



02 문제

운영 비용상 **GPT Mini** 급 **저렴한 AI**를 써야 함 (표준 대비 **6-10배** 저렴)
→ 그러나 싼 모델일수록 커지는 **비결정성을 최소화**해야 함



03 랄프 루프 목표

품질 평가 케이스 221개 × 10회 실행 → 9회 이상 성공 시 PASS (총 2,210번)
→ 이 목표를 달성할 때까지 **프롬프트(APO)** 와 **Pre/Post 프로세싱** 을 반복 수정 (안정성 위해 연속 3번 통과)

💡 **APO** = Automatic Prompt Optimization · 매 루프가 프롬프트와 전·후처리를 자동으로 다듬어 품질을 끌어올림

DAY 3

랄프 루프 엔지니어링

직접 만들며 익히는 핵심 기법 4가지

랄프 루프의 엔지니어링적 가치

AI 엔지니어링 측면에서 중요한 개념들을 내포 - **직접 만들 때 이해하고 활용할 4가지**

01  독립 컨텍스트 활용

02  세션·이터레이션 간 컨텍스트 공유

03  백그라운드 & 병렬 서브에이전트

04  정량적 목표 지표

👉 그래서 오늘 **첫 번째 실습은 랄프 루프를 직접 구현**해보는 것

독립 컨텍스트 활용 · 메인 vs 서브 컨텍스트 비교

그냥 이터레이션을 반복하면 컨텍스트가 짹 차고 품질·성능이 급락 - 이터레이션별로 컨텍스트를 분리해 독립적으로 유지

메인에서 계속 실행

메인 컨텍스트 (이터마다 계속 누적)

메인 이터1

메인 이터2

메인 이터N

빠르게 차다가 → N에서 짹 참

이터레이션
분리 →

서브에이전트로 분리

이터레이션마다 - 메인(거의 안 참) vs 서브(30-40% 후 소멸)

메인 이터1

서브-아터1

소멸

메인 이터2

서브-아터2

소멸

메인 이터N

서브-아터N

소멸

독립 컨텍스트 활용 · 독립 컨텍스트를 유지하는 두 가지 패턴

컨텍스트를 독립적으로 유지하려면 실행 자체를 메인 밖으로 분리해야 함 - 두 가지 방법

패턴 ①

Headless

스크립트로 클로드 코드를 **Headless**로 실행

며칠 돌릴 것 같으면 필수

패턴 ②

서브에이전트 권장

모든 인터레이션을 **BG 서브에이전트**로 위임하고 결과값만 메인
이 수신

약간씩 쌓이지만 실용적으로 문제 없음

△ 참고 엔트로픽이 Headless에 별도 과금·민감 반응이 있어 현재로선 서브에이전트 방식 추천

독립 컨텍스트 활용 · 클로드 코드 goal - 랄프 루프와 닮은꼴

직접 만들기 전 - 클로드 공식 /goal도 목표 달성까지 스스로 반복함 (2026년 5월 출시 · v2.1.139+)



**랄프 루프**
컨텍스트 · 매 인터레이션 새 세션(독립)
독립 컨텍스트·이력 주입을 내가 통제

VS

**/goal**
컨텍스트 · 같은 세션 이어가며 누적
손쉽지만 같은 세션에 누적됨

✓ 좋은 조건 측정 가능한 종료 상태 + 증명 방법("npm test가 0으로 종료") + 지켜야 할 제약 (+ "또는 20턴 후 중단" 상한)

세션·이터레이션 간 컨텍스트 공유 ·누적 메커니즘

매번 독립 컨텍스트의 단점은 기존 이터레이션에서 배운 것이 활용되지 않는 것 - 그래서 배운 것을 기록해 다시 주입



💡 이력 로그(파일.git)가 이터마다 **한 칸씩 쌓여** 다음 이터레이션의 독립 컨텍스트에 다시 주입됨 ↻ - 워크플로우 설계에 특히 유용 (예: Day2 태스크 분할)

세션·이터레이션 간 컨텍스트 공유 · 응용 사례

배운 것을 쌓아 다시 쓰는 패턴은 워크플로우 설계에서 특히 강력 - 대표 사례 두 가지



긴 태스크를 작은 단위로 분할

긴 태스크 (한 번에 안 끝남)

↓ 작은 단위로 분할

세션 1 · 조각 A

세션 2 · 조각 B

세션 N · 조각 C

각 세션 = 독립 컨텍스트 + 공유 이력

큰 일을 작은 단위로 쪼개 여러 세션이 이어받아 해결 - 각 세션은 깨끗하게 시작하되 이전 이력을 넘겨받음



OS · 개인 · 팀 레슨 축적

OS

개인

팀

↓ 한곳에 축적

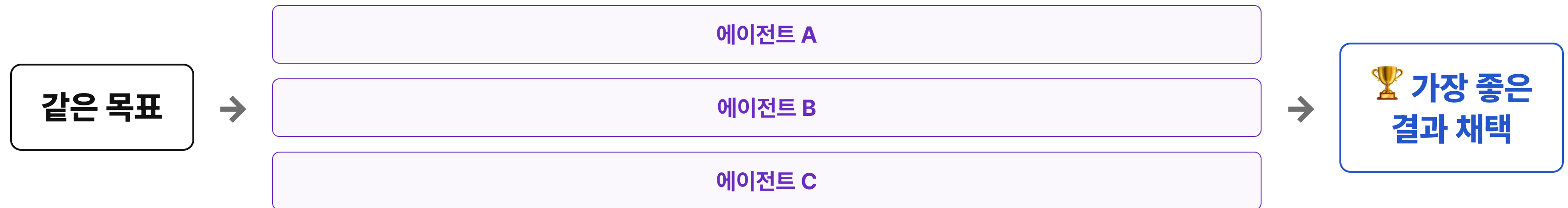
컨텍스트

모든 세션이 계속 꺼내 재사용

작업하며 얻은 레슨을 한곳에 쌓아두고 이후 모든 세션이 계속 참고 - 컨텍스트 엔지니어링의 핵심 자산

백그라운드 & 병렬 서브에이전트

서브에이전트를 BG로 실행하면 병렬로 실행할 수 있음



👉 대표적 활용법은 여러 에이전트에게 같은 목표를 준 후 가장 좋은 결과를 채택하는 것

목표 지표 결정 · 세 가지 유형 소개



앞의 셋이 인프라라면, 이것은 실제 결과의 품질을 결정짓는 가장 중요한 부분



정량적

오류 0개 · 테스트 모두 PASS · 정확도 90% 이상



체크리스트

모두 통과 혹은 n개 이상 통과



루브릭 rubric

정성 평가를 정량화 - 사람 혹은 **AI가 채점** (LLM as a Judge)

👉 비결정성 면에서 **사람보다 AI 채점이 낫다**는 평가도 있음 특히 Opus 4.7

목표 지표 결정 · 루브릭(rubric)이란 - LLM as a Judge



정성적 품질을 점수로 환산 - 평가 관점을 여러 개 정의하고 사람 혹은 AI가 채점 (AI가 채점하면 **LLM as a Judge**)



강점 = 재현성

사람 대비 강점은 '정확성'이 아니라 **재현성** - 같은 답에 같은 점수를 일관되게 줌 (특히 Opus 4.7+)



신뢰도 확보

단일 judge는 흔들리고 속을 수 있음 → **다관점 루브릭 + 여러 번 채점 (pass@N)**

⚠ 편향 주의 - 긴 답·앞선 답을 선호하는 **편향은 별도로 잡아야 함**

목표 지표 결정 · 루브릭 예시 - UX 평가 (LLM as a Judge)



평가 관점	1점	3점	5점
인지 부하	한 화면에 10개 이상 필드, 그룹화 없음	5~7개 필드, 일부 그룹화	3개 이내 또는 단계 분할, 우선순위 명확
에러 회복 가능성	"잘못된 입력입니다" 일반 메시지	필드 위치는 표시, 원인은 모호	위치 + 원인 + 수정 방법까지 한 번에
신뢰감 신호	왜 필요한지 설명·정책 링크 없음	정책 링크 존재, 사용처 설명 부재	민감 필드 옆 "왜 필요한지" 1줄 + 정책 링크
다음 행동 명확성	제출 버튼만, 이후 흐름 미언급	"가입 완료" 안내, 이후 흐름 모호	"인증 메일이 발송됩니다" 같은 사전 예고

목표 지표 결정 · 루브릭 예시 - Sellop 스펙 인터뷰 (게이트형)



UX 루브릭과 달리 - 1~5점 단순 척도가 아니라 **7개 차원 × 게이트** 구조. AI가 기획 인터뷰를 진행하며 매 턴 채점 → 모두 통과해야 종료

문제 Problem

깊이 Depth

일관성 Coherence

실현성 Feasibility

단순성 Simplicity

근거 Evidence

완료 기준 Acceptance



게이트 = 최저 차원

가중합이 아니라 가장 낮은 차원이 결정 - 한 차원이라도 **95점 미만이면 미통과** (평균으로 약점을 못 가림)



Q&A

주제 - 랄프 루프 엔지니어링

랄프 루프 직접 만들기

PRACTICE

다양한 엔지니어링을 활용해 나만의 랄프 루프 스킴 제작  **25분**

 **목표** 내 OS에 직접 랄프 루프를 만들며 다양한 엔지니어링 적용해보기

01 목표 & 측정 지표를 파라미터로 받는 랄프 루프 스킴 만들기
· 예: "E2E 전체 수행 시간을 5분 이하로 줄여줘"

02 각 인터레이션이 독립 컨텍스트로 실행되게 만들기
· Headless 혹은 서브에이전트

03 (선택/도전) 인터레이션에서 배운 것을 누적 기록해 이후 인터레이션에서 참고
· 예: lessons.md에 기록 → 다음 실행 시 참고

04 랄프 루프를 실제 사용해보고 예상대로 동작하는지 관찰

REVIEW

랄프 루프 직접 만들기 나누기

 **10분**

1인 2분 다른 사람의 의견도 들을 수 있도록 배려 🙏

SHARE 01

독립 컨텍스트를 만들기 위해 어떤 방법을 사용했나요? 왜 그 방식을 선택했나요?

SHARE 02

세션·인터레이션 간 컨텍스트 공유는 어떤 방식으로 했나요?

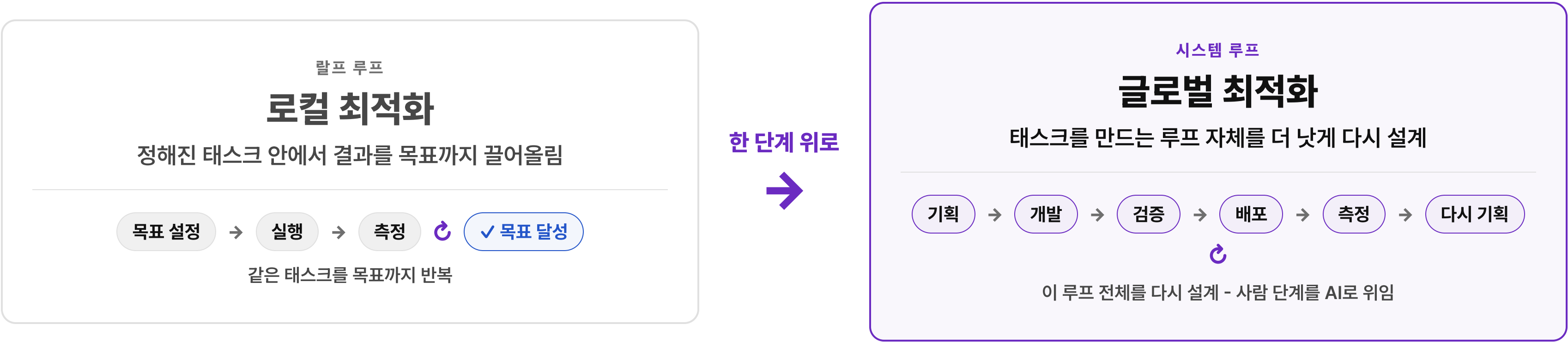
DAY 3

시스템 루프

로컬 최적화에서 글로벌 최적화로

시스템 루프란

우리가 반복하는 작업 환경(예. 워크플로우) **자체를 개선하는 루프** - 우리 일 대부분은 반복 루프임



루프 예시



🔄 10단계 측정 결과로 다시 1단계 기획 - 루프 반복

루프를 보며 해볼 만한 질문

01 **자동화 추구** - 자동화할 수 있는가?

자동화할 수 있다면 하는 게 좋다

02 **사람의 역할** - AI가 할 수 있어도 사람이 끝까지 잡아야 할 것은?

모든 것을 위임하는 게 능사는 아님

03 **적극적 위임** - 위임 못 한다면 그 이유는? 신뢰? 정책? 모호함?

AI 리더십 부족이거나, 스스로도 정리가 안 됐거나

04 **지속적 개선** - 루프가 돌 때마다 조금씩이라도 발전하는가?

0.01%라도 개선되면 복리의 힘

DAY 3 · 시스템 루프

자동화 추구

사람의 한계 - 자동화할 수 있는 건 과감히 OUT으로

사람의 한계

루프 관점에서 **사람이 병목이 되기도 함**

01 **시간 · 속도** - 일하는 시간과 속도에 제한, 사람이 병목이 됨

예. PR 후 사람의 코드 리뷰 병목

02 **일관성** - 비결정적이라 AI 대비 일관성이 부족할 수 있음

예. 루브릭 평가를 사람이 할 때 기준이 왔다갔다

👉 다수가 하기 싫어하는 **문서 업데이트·기계적 리뷰** 등이 특히 좋은 자동화 후보

IN → ON → OUT

원래 제어공학·사이버네틱스 용어 - 자동 루프 안에서 **사람이 개입하는 지점**



Human **IN** the loop

매 결정에 직접 참여

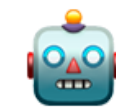
자율주행 LV 1-2



Human **ON** the loop

감독·거부권만

자율주행 LV 3-4



Human **OUT** the loop

완전 자율

자율주행 LV 5

사람이 더 개입

자동화가 더 강함

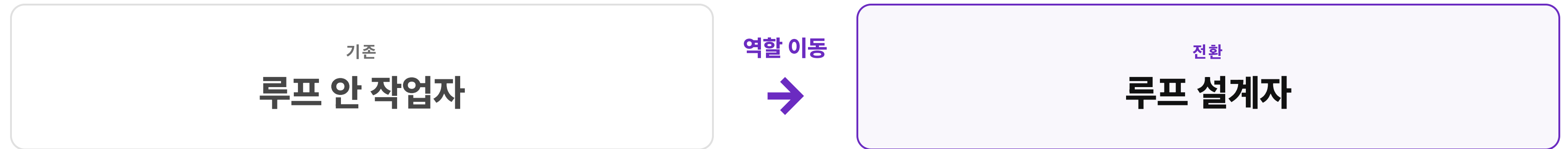
👉 자동화가 강해질수록 사람은 **IN → ON → OUT** 으로 물러남 - 자동화할 수 있는 지점은 과감히 **OUT** 쪽으로

DAY 3 · 시스템 루프

사람의 역할

AI가 해도 사람이 끝까지 잡아야 할 것

루프 설계자로서의 역할



🎯 **이상적 목표** - 특정 범위 루프 안의 일은 모두 자동화하는 것

🧭 사람의 루프 안 개입을 최소화하려면 **종료조건을 포함한 의도·정보**를 잘 정리해야 함

👉 그래서 **컨텍스트 엔지니어링(Day2)**이 루프의 전제였던 것

루프 안팎의 일부 작업을 담당

현실적으로 모든 것을 위임하는 게 능사는 아님, 아래는 예시

01



무엇을 · 왜

AI는 '어떻게'에 강하지만 '무엇을 왜'는 사람이 정의

02



성공의 기준 · 종료 조건

무엇이 '잘된 것'인지 정하는 것 (검증의 출발점)

03



가치 판단 · 최종 책임

되돌리기 어려운 결정, 윤리·책임이 걸린 최종 승인

04



오프라인과의 상호작용

로봇이 빠르게 발전 중이지만 당분간은 사람이 담당

👉 단, AI 발전에 따라 이 경계도 바뀔 수 있음 (예. AI가 검증도 더 잘하게 됨)

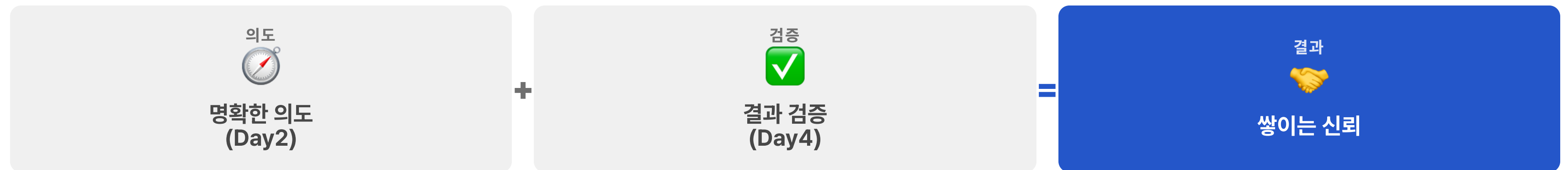
DAY 3 · 시스템 루프

적극적 위임

신뢰는 감정이 아니라 구조다 - 신뢰 · 의도 · 검증

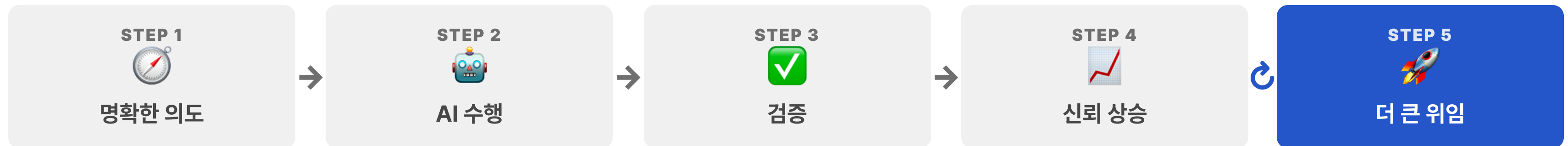
더 많이 위임

더 많이 위임하려면 신뢰가 필요함 - 그런데 **신뢰는 막연한 믿음이 아님**



👉 위임 못 하는 이유 대부분은 신뢰가 아니라 **의도·검증의 결핍** - 의도가 모호하거나, 검증 장치가 없거나

신뢰 · 의도 · 검증 선순환 루프



👉 이 자체가 하나의 루프 - 한 바퀴 돌 때마다 위임 범위가 넓어지고, 신뢰도 복리로 쌓임 (지속적 개선과 같은 원리)

위임 못 하는 이유, 다시 묻기

판단력 부족

→ 컨텍스트를 충분히 주고 있는가? **대개 신뢰가 아니라 컨텍스트 문제**

실수

→ 사람도 실수함 · 기존엔 어떻게 관리했나? **되돌릴 수 있게 만들면?**

검증 불가

→ 정량 지표·루브릭 등으로 자동화할 수 있나? **못 만들면 그게 진짜 병목**

책임·규제

→ '결정'은 AI가, **'최종 승인'만 사람으로 분리**하면 되지 않을까?

👉 단, AI가 의도를 배신할 때가 있음 (**치팅·포기·월권**) - 그래서 검증·하네스가 필요 → **Day4**

DAY 3 · 시스템 루프

지속적 개선

자율 개선 루프 - 개선조차 AI가 하게 만들기

왜 지속적 개선인가

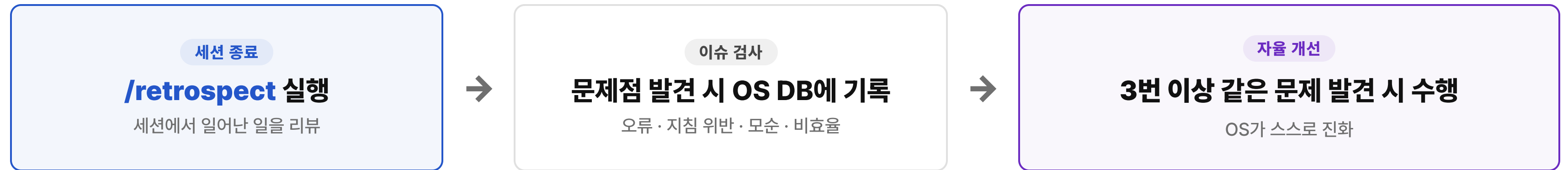
01 우리는 현재까지 루프 안에서 일했고, 또 루프를 돌 것임

02 루프를 돌 때마다 뭔가 조금씩이라도 개선되면 좋지 않을까?

한 걸음 더

그리고, 개선조차도 AI가 할 수는 없을까?

OS 자율 개선



측정 지표

OS 건강도 = $1 - (\text{이슈 발생 세션} / \text{전체 세션})$

OS가 건강도를 높이는 방향으로 스스로 진화함

컨텍스트 · 스킬 자율 개선



컨텍스트 자율 개선 · /ssot

의도를 새로 인터뷰하거나 코드를 수정한 후 **/ssot** 수행 - 변경을 바탕으로 컨텍스트 전체를 심층 탐색·갱신

컨텍스트가 항상 날카롭게 유지됨

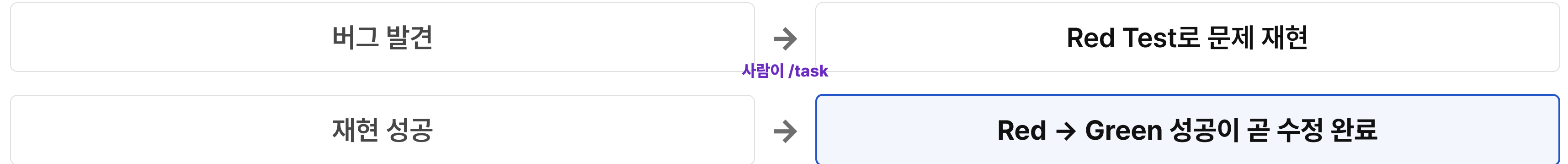


스킬 자율 개선 · Gotcha

/retrospect가 영구 저장할 지식을 검사해 **스킬 하단 Gotcha 란에 기록** - 스킬에 경험치가 쌓이며 더 똑똑해짐

스킬별로 측정하진 않고 OS 건강도를 봄 (tradeoff)

(반자율) Red Test First



👉 버그가 발견돼도 **바로 고치지 않고**, Red Test로 재현에 성공해야 고침 - **문제를 재현하는 것이 확인된 값진 테스트 확보**

💡 뮤테이션 점수 = 잡은 뮤테이션 / 심은 뮤테이션 stryker · PR / Nightly에서 측정

DAY 3 · 시스템 루프

루프 사례 소개

스펙 구현 루프

디자인 목표

01 🙄 가시성

수행 중인 태스크에 **언제든 질문**할 수 있어야 함

02 🌐 접근성

외부에서도 원할 때 **AI와 협업**할 수 있어야 함

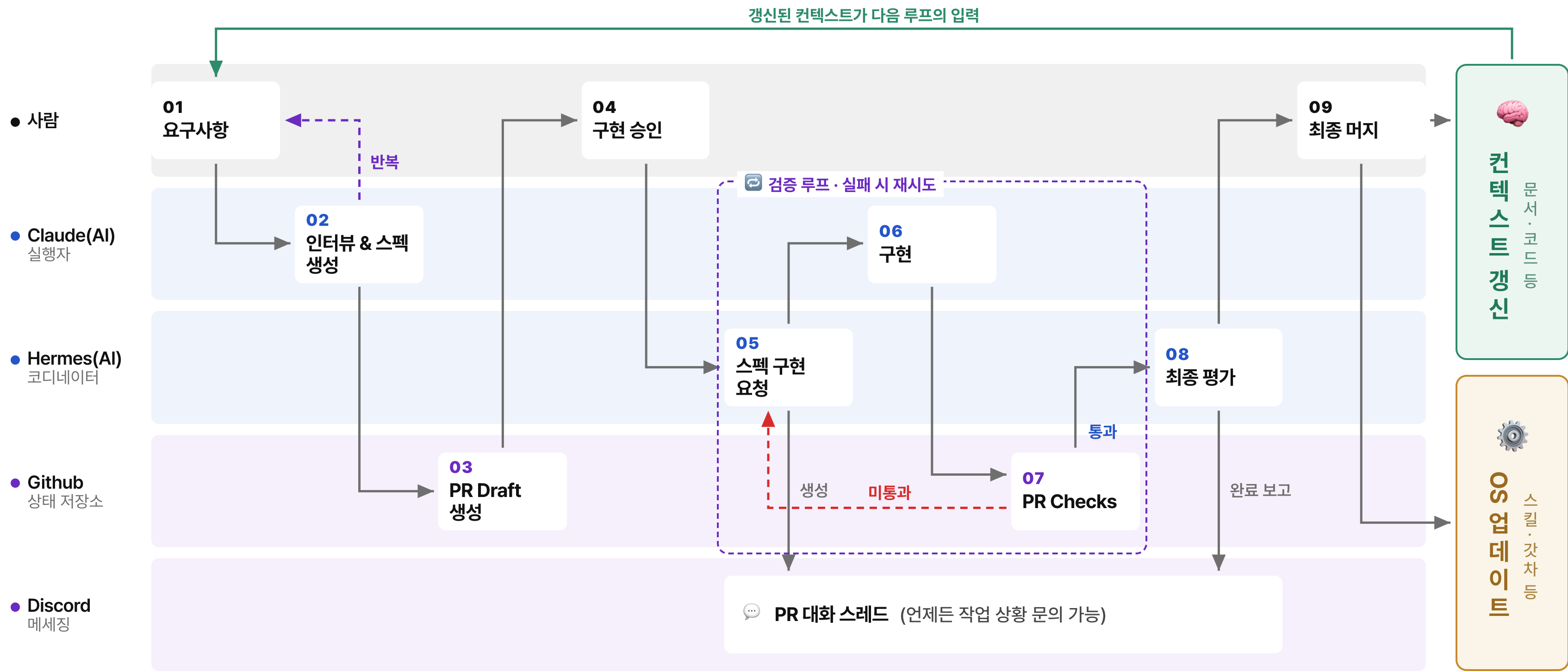
03 📖 기록성

증발되기 쉬운 수행 이력을 **기록·활용**할 수 있어야 함

04 ✅ 완결성

사람 개입 없이 **끝까지 완료**해낼 수 있어야 함

스펙 구현 루프



Github PR에 스펙 박제



MinCha commented 10 hours ago

...

plan: -
mode: REFACTOR
caller: spec-interview

E2E 집합 보존(set-preservation) 단언 패턴 도입

1. 상태

계획

2. 목표

어떤 요소에 행위한 뒤 그 요소만 단언하지 말고, 같은 화면의 건드리지 않은 형제 집합이 행위 전 그대로 보존됐는지 DOM과 DB를 교차해 함께 단언한다. "신규 상품 카드 소멸" 부류(PR#6, 5번째 변종) 회귀를 E2E 단계에서 잡는 것이 목표.

3. 기대 효과

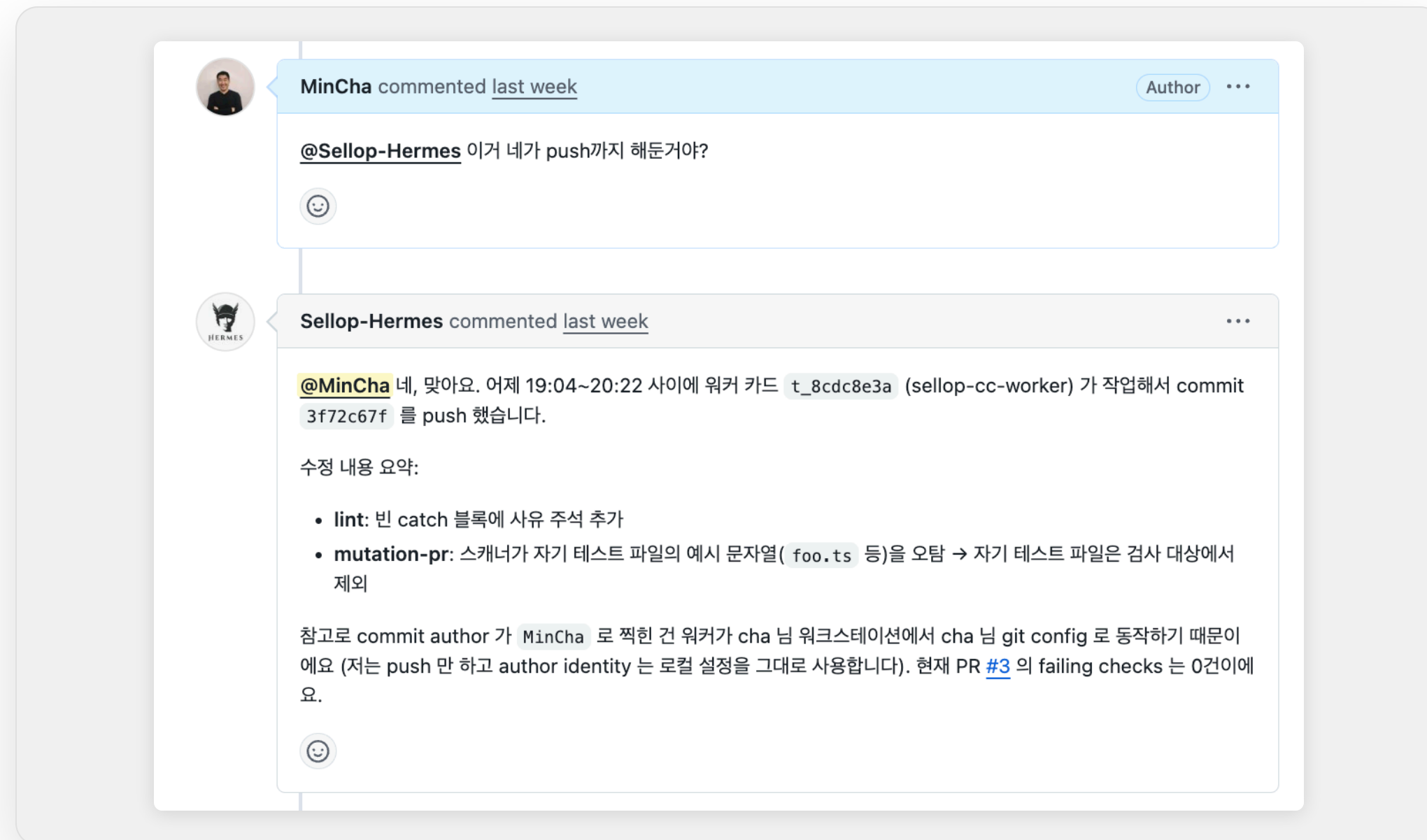
빌더 프리뷰에서 한 상품만 손대면 나머지 상품 카드가 화면에서 사라지던 회귀가 반복돼 왔다. 기존 검증은 건드린 항목만 확인하고 형제 집합의 보존은 보지 않아, 사용자가 본 화면과 서버 진실이 어긋나도 통과시켰다.

항목	전	후
형제 상품 보존	건드린 상품만 단언, 나머지 집합은 미검증	화면(DOM)과 서버(DB)의 상품 집합을 교차해 보존을 함께 단언
화면-서버 어긋남(stale 화면)	새로고침하면 복구돼 테스트가 못 잡고 통과	어긋나는 순간 그 자리에서 단언 실패로 드러남
신규 상품 카드 소멸 회귀	E2E가 false-pass, 사용자만 발견	신규 상품 생성 흐름에서 단언 실패로 사전 차단

02-03 · 스펙 생성 → PR Draft

- 스펙(태스크 문서)이 **PR Draft 본문에 그대로 임베딩**됨
- 작업 단위가 **PR로 추적**되어 진행.이력이 남음

PR 덧글로 Hermes와 대화



04-05 · 승인 → 스펙 구현 요청

- PR 덧글로 "구현하라" 진행 승인
- Hermes가 덧글을 인식해 구현을 진행하고 경과를 보고

Discord 스레드에서 문의



Discord 레이어 · 가시성

- 태스크별 **PR 대화 스레드**로 진행 상황을 실시간 보고
- 자리에 없어도 **Hermes**에게 언제든지 작업 상황 문의



Q&A

주제 - 시스템 루프

시스템 루프 설계하기

PRACTICE

내 일의 반복 구조를 루프로 보고
스스로 나아지는 장치 설계

 25분

 목표 내 일의 반복 구조를 루프로 보고, 흐름을 설계해보기

01 내 OS(또는 팀)의 핵심 루프 그려보기

- 단계로 분해 + 각 단계를 누가 하는지 표시 (사람 / AI / CI)
- 병목·반복·낭비가 생기는 지점 1곳 찾기

02 (선택/도전) 루프 안에 자율 개선 장치 설계해보기

- 어디서(언제) / 무엇이 / 어디에 쌓이게 할지 정하기
- 자율 탐지형(스스로 찾아 고침) vs 자산 축적형(겪은 걸 남김) 중 택1

03 (선택/도전) 개선을 나타내는 정량 지표 설계해보기

- 한 줄 수식으로 표현 (예: 건강도 = 1 - 나쁜 사례/전체)
- 무엇이 측정되고, 높/낮음이 무엇을 뜻하는지 명확히

REVIEW

시스템 루프 설계하기 나누기

 15분

1인 3분 다른 사람의 의견도 들을 수 있도록 배려 🙏

SHARE 01

어떤 루프를 그렸고, 병목은 어디였나요?

SHARE 02

자율 개선을 어디에 심었고, 무슨 지표로 측정하나요?

오늘 우리가 완료한 것



랄프 루프 직접 구현

유용한 엔지니어링을 직접 만들며 경험함



시스템 루프 관점 획득

내 일을 루프로 바라볼 수 있는 시각을 얻음

이제 할 수 있는 것

내 일을 루프로 보고, 개선 장치와 측정 지표를 직접 설계할 수 있음

더 많이 위임하려면

더 많이 위임하려면 **AI에 대한 더 큰 신뢰**가 필요함 - 그러나 때때로 AI는 내가 원하는 방향으로 움직이지 않음



치팅

겉에만 그럴듯하게 만들고 실제 업무는 완료하지 않음



포기

정해진 워크플로우를 준수하지 않고 중간에 포기함



월권

정해진 권한을 어기고 손대지 않아야 할 파일을 수정함

👉 4일차에는 이러한 AI를 제어하는 **하네스**에 대해 다룰 것

내 OS에 시스템 루프 구축하기

💡 강사 권장

루프는 OS가 스스로 반복하고 나아지게 만드는 엔진이고, 반복을 설계하는 사람이 AI를 이끕니다. 오늘 배운 랄프 루프와 시스템 루프를 실제 내 OS에 심어, 작게라도 실행할수록 조금씩 나아지거나 자산이 쌓이는 구조를 만들어보세요.

완료 조건

필수

- 1 실행할수록 개선되는 시스템 루프 1개 구현**
작은 루프여도 무관, 수행될 때마다 작게라도 개선되거나 축적되는 루프를 구현
예: 세션 끝에 /retrospect로 이슈 기록·개선, lessons.md에 배운 것 누적
- 2 만든 시스템 루프를 문서화해 1P로 정리**
루프의 핵심 흐름이 한눈에 보이게 1페이지로 정리해 파일로 남김 (PR 본문에 첨부)
문서화하면서 눈에 띄는 아쉬운 점이 있으면 그 자리에서 개선해보기

도전

- 1 시스템 루프가 나아지는지 측정 지표·루브릭 추가 (필수1과 연결)**
내 시스템 루프가 나아지고 있는지를 재는 정량 지표 또는 루브릭 정의
예: 한 줄 수식 건강도 = 1 - 나쁜 사례/전체, 또는 여러 관점의 루브릭
- 2 5차원 루브릭 + 랄프 루프 2h+ 스냅샷**
5차원 이상의 루브릭과 함께 랄프 루프를 2시간 이상 돌려보고, 각 이터레이션별 결과
물이 어떻게 변해가는지 스냅샷으로 남기기

과제 TIP

💬 PR을 할 때 접근 방식 · 설계 의도 · 궁금한 점을 함께 적어주시면 더 양질의 리뷰를 드릴 수 있습니다!

🚀 4주 동안 OS를 만들고 · 사용하고 · 개선 할수록 강의에서 더 많은 것을 얻어가실 수 있습니다!